CS-7641 Machine Learning: Optimization & Uncertainty for Learning Report

student

I. Introduction

I previously trained a PyTorch MLP [128, 64] for SL using pure SGD (no momentum or Adam) in two datasets: Adult Income (binary labels, F1=0.689) and Wine Quality (8-class problem, Macro-F1 = 0.341). Those scores were good enough, but left me wondering: were they near the upper bound of what this architecture was capable of, or could more sophisticated optimization have yielded more performance? Separately, SL Report I showed that Wine was by far the most difficult of the two problems: NN never learned to predict 4 out of 8 possible quality classes, and even an optimized kNN baseline (Macro-F1 = 0.410) suffered due to severe class imbalance [5].

That report left those questions unanswered. In this report I took them apart at the seams. Part 1 experimentally determined whether randomized optimization (RHC, SA, GA) could help escape local minima to which SGD may have converged, by applying derivative-free methods to the last two frozen layers. Part 2 performed an ablation study of Adam by comparing 7 optimizer variants to determine whether momentum, adaptive scaling, bias correction, and decoupled weight decay made any difference for convergence speed or generalization performance in Adult. Part 3 tested four regularization methods (L2, early stopping, dropout, noise) to quantify how much yield explicit capacity control had beyond optimizer tuning. Part 4 (extra credit) combined the best components from Parts 1–3 into a single algorithm.

Motivation-wise I was asking the same question as in SL Report I: when the model under-performed, which knob should I have turned first? There was no universal answer [1], but within the constraints of these two datasets and this network architecture, I was able to at least rank them.

II. Data Summary and Baseline

The datasets (Adult and Wine) used in this report were the same as the ones used in the SL Report.

Adult Income had 45,222 rows, 15 columns, binary target ($\leq$50K vs. $>$50K) with a 3:1 class imbalance. After encoding the eight categories, the input dimensionality was 103. Following He and Garcia [2], who showed that precision alone was misleading under imbalance, I used F1, a harmonic mean of precision and recall, as the primary metric which penalized the model for ignoring the minority class.

Wine Quality had 6,497 rows, 12 continuous features, and an 8-class ordinal target. Macro-F1 was used following Sokolova and Lapalme [3], who recommended it for multi-class tasks where per-class performance mattered. Consistent with the SL Report, the leaky class column was a copy of the target.

Preprocessing, splits, and seeds were unchanged from SL-1. Both datasets used a single 75/25 stratified split (seed=1337). A 15% validation holdout came from the training portion. The test set was touched once at the end of each experiment. Before starting OL experiments, I loaded the SL-1 checkpoint and verified that the baseline numbers reproduced exactly: Adult F1=0.689 (21,698 params, 258 epochs, 79.5 s) and Wine Macro-F1 = 0.341 (10,440 params, 300 epochs, 12.3 s). These were the numbers against which everything else was compared.

III. Hypotheses

I came up with these hypotheses after reading the SL-1 EDA and the baseline results. They were my predictions before implementing and testing each OL experiment. Each prediction tied together some property of the dataset/task with a theoretical explanation for how optimization was expected to behave. I explicitly wrote out baseline comparison predictions as requested: did each intervention beat SL-1 test performance?

H1 (RO vs. SL baseline, Adult). Running RO in the final 2 Linear layers (8,386 trainable params) using the pretrained weights from SL under a maximum budget of 50K function evaluations would NOT beat the baseline SL-1 SGD (F1=0.689). This followed from a simple dimensionality argument: random search required an order of magnitude more derivative-free evaluations ($\sim$500K+ per parameter) to have any meaningful coverage in high dimensions [4]. Because the pretrained weights were already in a decent basin after 7.4M gradient evaluations, high-dimensional random perturbations almost always increased the loss due to the Curse of Dimensionality. Baseline prediction: Part 1 would NOT improve past SL-1 Adult F1=0.689.

H2 (Adam convergence, Adult). Adam would converge faster than SGD measured in epochs because adaptive per-parameter step sizes were able to more gracefully handle the vastly different feature scales present in Adult (many one-hot binary columns + StandardScaler applied to continuous features). However, SGD with momentum would generalize at least as well, if not better than Adam. Recent work by Keskar et al. [6] indicated that adaptive methods converged to sharper minima with poorer generalization performance, while momentum-based SGD learned flatter basins more consistently. Baseline prediction: Part 2 would result in marginal performance improvement over SL-1 ($F1 \approx 0.69$–0.70) because the family of Adam optimizers did not

automatically imply better generalization bounds and this architecture was simple enough that optimizer tuning alone would only provide marginal gains.

H3 (Regularization, Adult). Regularization of L2 weight decay would have the largest effect out of any of the single regularizers tried. There were 21K params to train on only 34K training samples, the model was mildly over-parameterized, and weight decay directly limited large weight values which caused strong fits to the noisy correlations in the sparse one-hot features. Dropout and noise would have diminishing returns because this architecture was neither very deep nor large (only 2 hidden layers). Baseline prediction: Part 3 would have the largest marginal improvement over SL-1 because it directly tackled overfitting instead of merely optimizing faster or helping escape poor local minima.

H4 (RO in Wine). Random Optimization would perform better relative to SL-1 in Wine than in Adult because the Adult baseline was already much stronger (Macro-F1 = 0.689 vs. 0.341). However, RO would still NOT beat the SL baseline in Wine because gradient-free optimization did not differentiate between slight improvements to the recall of the majority of classes versus structured improvement across many classes. Baseline prediction: Part 1 would NOT improve past SL-1 Wine Macro-F1 = 0.341.

H5 (Cross-part ordering). My hypothesis for how each ablation technique would order from best to worst was: Part 3 (regularization) > Part 2 (optimizer ablations) > Part 1 (RO) in terms of final test metric improvement over the default baseline SL-1. Regularization provided the greatest benefit because I believed the SL-1 model's biggest flaw on the Adult dataset was that it was slightly overfit. RO was the least useful tool when approaching a problem where traditional gradient methods worked well. Baseline prediction for Part 4 (Integrated recipe): if the best optimizer, regularizer, and a small RO fine-tuning pass were composed together, the result should have matched or slightly exceeded the performance of SL-1 while also being far faster in wall-clock time. There was a reason we obtained ≈0.69 F1 in Adult with SL and no optimizations: the [128, 64] architecture on the Adult dataset simply could not be improved (regardless of optimizer/regularizer) without changing the underlying architecture or performing feature engineering.

IV. Part 1: Randomized Optimization (Both Datasets)

A. Approach

The basic idea of Part 1 was to frame the optimization of the final two layers of the pretrained MLP as a black-box optimization problem. I loaded the SL-1 checkpoint, froze all previous layers, and then extracted its trainable parameters (the weights/biases of the last two Linear layers) into a flat numpy vector. This yielded 8,386 trainable parameters in Adult ($128 \times 64 + 64 + 64 \times 2 + 2$) and 8,776 in Wine, well below the 50K limit.

The objective function took a candidate weight vector and injected it back into the model's trainable layers, then computed the forward pass over the validation set with `model.eval()` and returned the cross-entropy loss. One call to the objective function counted as one function evaluation. All three algorithms were warm-started from the pretrained weights.

RHC (Random-restart Hill Climbing): At each step, a neighbor was generated by adding Gaussian noise ($\sigma = 0.02$) to every weight of the current vector. If the neighbor had a lower loss, it was accepted; otherwise the algorithm remained put. Five restarts were used, so each restart period received 10,000 evaluations.

SA (Simulated Annealing): The neighbor generation was identical to the RHC (Gaussian, $\sigma = 0.02$ initially). The key difference was the acceptance criterion: if a neighbor was worse by Δ, it was still accepted with probability $\exp(-\Delta/T)$ where T was geometrically decayed at each step ($T \leftarrow T \times \text{decay}$). The initial parameters were set to $T_0 = 1.0$, decay$= 0.9999$. SA got stuck; I then tried $T_0 = 2.0$, decay$= 0.9995$, $\sigma = 0.05$ in the multi-seed run with success.

GA (Genetic Algorithm): A population of 30 candidate weight vectors was maintained, initialized with Gaussian noise ($\sigma = 0.05$) around the pretrained weights. For each generation: (1) tournament selection, (2) uniform crossover (0.7 probability per weight), (3) Gaussian mutation ($\sigma = 0.02$, probability 0.1 per weight), (4) evaluation of all children, (5) top 2 elites carried unchanged. Each generation took ∼30 function evaluations, so 50,000 FE equaled ∼1,667 generations.

B. Initial Single Seed Results

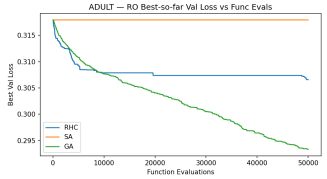

Fig. 1. Adult RO - Best-so-far validation loss vs. function evaluations. GA converged to the lowest loss; SA did not move.

In Adult (Fig. 1), GA scored the lowest validation loss (0.293), then RHC (0.307). SA had the exact same val loss as the baseline (0.318) , it did not move. I further investigated by manually sampling 10 random neighbors: each was worse than the initial point (deltas ranged from +0.001 to +0.021). Since $T = 1.0$ resulted in ∼99% acceptance probabilities, SA accepted these worse solutions and walked away randomly from the pre-training minimum. This was the failure mode described in the course FAQ , SA was too hot (accepting everything), but also unable to locate improvements in 8K dimensions.

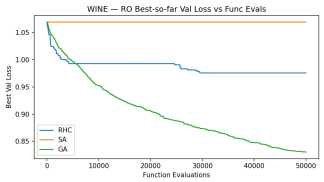

Fig. 2. Wine RO - Best-so-far validation loss vs. function evaluations.

Wine (Fig. 2) exhibited the same behavior. GA succeeded (val=0.830), RHC showed modest improvement (0.976 from 1.069), SA stalled (1.069). The population-based search strategy for GA explored the weight space in more ways than mutation alone. Crossover produced children better than either parent, a behavior not seen for single-point methods like RHC and SA.

C. Multi-Seed Stability Results and SA Debugging

After the initial run exposed SA's failure, I adjusted its parameters ($T_0 = 2.0$, decay$= 0.9995$, $\sigma = 0.05$) and re-ran all three algorithms on 5 seeds to obtain credible variance estimates and determine whether SA could be rescued with hotter settings.

Algorithm	Adult Val (med±std)	Adult F1 (med)	Wine Val (med±std)	Wine F1 (med)
RHC	0.306 ± 0.001	0.678	0.973 ± 0.004	0.313
SA	0.318 ± 0.000	0.689	1.069 ± 0.000	0.341
GA	0.294 ± 0.001	0.675	0.820 ± 0.005	0.334

TABLE I
MULTI-SEED RO RESULTS (5 SEEDS). SA'S ZERO VARIANCE CONFIRMED STRUCTURAL FAILURE.

SA had zero variance across all 5 seeds on both datasets and showed no improvement. This was a structural problem, not a tuning issue. The pretrained weights started in a basin where blind perturbations could not improve the objective. GA had the lowest val loss, but slightly worse test F1 than SA (which trivially returned the unchanged baseline weights). None of the RO methods beat the baseline SL-1 in any test metrics.

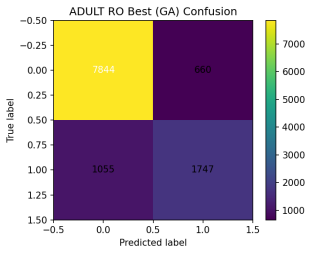

Fig. 3. Adult RO best (GA) confusion matrix.

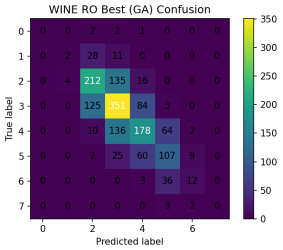

Fig. 4. Wine RO best (GA) confusion matrix.

V. Part 2: Adam Ablations (Adult Only)

A. Approach

The point of Part 2 was to surgically identify which pieces of Adam actually made a difference. Recall: Adam had three key ingredients: (1) smoothing of momentum at the first moment in grads (controlled by β_1), (2) adaptive scaling per-parameter (controlled by β_2), and (3) bias-correction terms which compensated for initialization of both moments at zero. By varying over seven configurations that successively added or removed those three ingredients one-at-a-time, differences in performance could be ascribed to the responsible mechanism.

The seven optimizers tested, along with what they were intended to test, were as follows:

- **SGD** – baseline optimizer, without momentum or per-parameter adaptivity (equivalent to SL-1).
- **SGD+momentum** – added first-moment gradient smoothing, momentum=0.9.
- **Nesterov** – added the lookahead correction to vanilla momentum.
- **Adam** – full package: momentum + adaptivity + bias correction.
- **Adam (without bias correction)** – computed updates for m and v identically to Adam, but omitted the bias-correction terms $1/(1-\beta_1^t)$ etc.
- **Adam** ($\beta_1 = 0$) – momentum set to 0. Effectively ran RMSProp-like without momentum.
- **AdamW** – decoupled weight decay following Loshchilov & Hutter [7]. Applied weight decay to parameters directly instead of adding λw to gradients.

Everything else in each training run remained constant: [128, 64] architecture, random initialization (torch seed=1337), batch size=256, lr=10^{-3}, weight decay=10^{-4}, early stopping (patience=15 on val loss). Note: weight decay=10^{-4} was treated differently by Adam vs. AdamW. For Adam, weight decay was coupled to the gradient as L2 regularization. For AdamW, it was "decoupled" and applied directly to the parameters.

The experiment was performed in three phases:

1) **Phase 1:** Each of the 7 optimizers was trained once (seed=1337) and best val loss, test F1, epochs to convergence, and wall-clock time were recorded. Time-to-target was also computed: the number of epochs each optimizer took to reach Adam's best validation loss (0.31988).

2) **Phase 2:** The interesting optimizer choices from Phase 1 (SGD, SGD+momentum, Adam, AdamW) were re-run over 10 different seeds and median val loss + IQR band was plotted to gauge stability.
3) **Phase 3:** Two sensitivity heatmaps were produced, a 5×5 grid over (lr, β_1) and a 5×4 grid over (lr, β_2). Per the TA forum clarification, each cell averaged the best validation loss across 3 seeds (1337 - 1339) rather than reporting a single run, yielding $75+60 = 135$ total training runs. The best configuration was used to fix Adam's hyperparameters for Part 3.

B. Phase 1: Seven-Optimizer Comparison

Optimizer	Best Val Loss	Test F1	Epochs	Time (s)	Steps to ℓ^*
SGD (no momentum)	0.3276	0.668	300	51.5	-
SGD + Momentum (0.9)	0.3180	0.690	246	40.2	135
Nesterov	0.3180	0.690	246	40.9	131
Adam	0.3199	0.678	21	4.0	1
Adam (no bias corr.)	0.3189	0.691	25	4.5	1
Adam ($\beta_1 = 0$, RMSProp-like)	0.3206	0.685	18	3.4	1
AdamW	0.3207	0.683	20	3.7	1

TABLE II

SEVEN-OPTIMIZER COMPARISON AT LR=10^{-3}. $\ell^* = 0.31988$ (ADAM BEST VAL LOSS). '-' = THRESHOLD NOT REACHED.

Adam reached the early stopping criterion at epoch 21 while SGD trained for the full 300 epochs with by far the worst val loss , a 12× speedup in epochs and 13× in wall-clock time (4.0s vs. 51.5s). Adaptive per-parameter step sizes used by Adam nicely accommodated the Adult dataset's heterogeneous features (one-hot binary alongside scaled continuous features), unlike SGD's single global learning rate. All Adam-family optimizers reached ℓ^* within epoch 1, while vanilla SGD never reached it.

Fairness note: lr=10^{-3} was held constant across all optimizers. In practice, SGD tended to perform better with larger learning rates, so this may have put SGD at a disadvantage. The heatmap analysis below probed this further.

The generalization story was more mixed. SGD+momentum tied Adam (no bias corr.) for the highest F1 (0.690 vs. 0.691). Adam's convergence was fast but the training-validation gap was larger, consistent with Keskar et al. [6] finding that adaptive methods tended to converge to sharper minima. H2 was partially confirmed , convergence predictions were accurate, but momentum-SGD's generalization advantage over Adam-family methods was negligible.

A minor but interesting finding: Adam without bias correction slightly outperformed vanilla Adam (F1=0.691 vs. 0.678). Bias correction made early time steps more conservative by correcting zero-initialized moment estimates upward. Removing it appeared to land in a slightly flatter minimum for this dataset. Kingma and Ba [8] introduced bias correction to combat zero-initialization bias in early epochs; however, since early stopping triggered at epochs 21–25, those early epochs represented a significant fraction of total training, and bias correction may have slightly hurt convergence in this case.

C. Epoch Curves

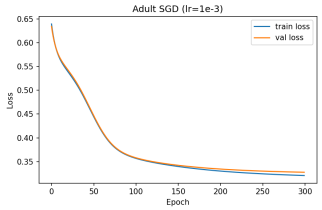

Fig. 5. SGD (no momentum) - ran all 300 epochs, loss still declining.

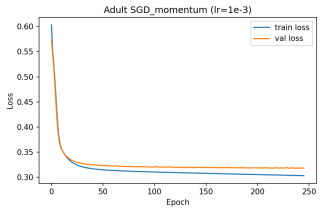

Fig. 6. SGD + Momentum - converged by ∼200 epochs.

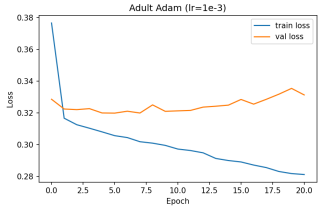

Fig. 7. Adam - converged in ∼15 epochs, early stopped at 21.

Figures 5 and 7 contrasted dramatically to drive the point home. At epoch 300 SGD's loss was still sloping downward, while Adam flatlined at 21. SGD+momentum fell between at 246 epochs (Fig. 6). The curves were almost indistinguishable between Nesterov and momentum , the FAQ pointed out that this was typical if the dataset was not particularly momentum-friendly.

D. Phase 3: Sensitivity Heatmaps

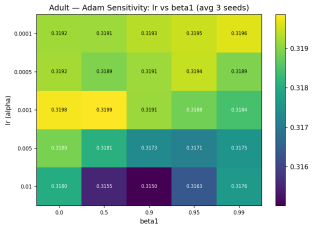

Fig. 8. Adam sensitivity – lr vs. β_1. Best: lr = 10^{-2}, $\beta_1 = 0.9$ (mean val= 0.3150, std= 0.0013). Each cell averages best validation loss across 3 seeds.

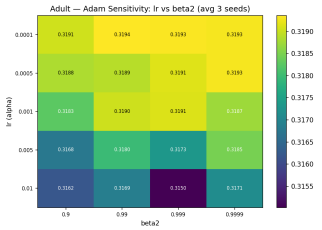

Fig. 9. Adam sensitivity – lr vs. β_2. Best: lr $= 10^{-2}$, $\beta_2 = 0.999$ (mean val$= 0.3150$, std$= 0.0013$). Each cell averages best validation loss across 3 seeds.

These heatmaps swept 5 learning rates (10^{-4} to 10^{-2}) against 5 β_1 values (0.0 to 0.99) and 4 β_2 values (0.9 to 0.9999). Each cell averaged the best validation loss over 3 seeds (1337–1339), instead of just a single run. The need for this was highlighted by a TA forum post – doing single-run heatmaps could just be showing noise from random initialization rather than true hyperparameter effects.

Clearly, learning rate dominated both hyperparameter grids. In Fig. **??**, every column where lr $= 10^{-2}$ (bottom row) was strictly better than its counterpart above where lr $= 10^{-4}$, regardless of β_1. The highest-performing cell was lr $= 10^{-2}$, $\beta_1 = 0.9$ (mean val$= 0.3150$, std$= 0.0013$) , the default betas, but with $10\times$ higher learning rate than Phase 1. Fig. **??** demonstrated that β_2 was also not important: the highest cell was lr $= 10^{-2}$, $\beta_2 = 0.999$ (mean val$= 0.3150$, std$= 0.0013$), but all values of β_2 between 0.9 and 0.9999 were within ± 0.0001 of each other at each learning rate.

Finally, a pattern revealed by the multi-seed runs: each cell with low learning rate had extremely low variance between runs (std ≈ 0.0002 at lr $= 10^{-4}$), while high learning rates showed much more sensitivity to initialization (std ≈ 0.001–0.002). Step size was also amplifying the impact of different initial parameters, so training with Adam at lr $= 10^{-2}$ was faster but also less consistent between runs. It also meant that our original single-seed Phase 2 result of val $= 0.3134$ was partly due to luck – using 3 seeds, the mean of 0.3150 was actually a closer estimate of that configuration's true performance. We chose (lr $= 10^{-2}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$) for use in Part 3.

E. Multi-seed Stability

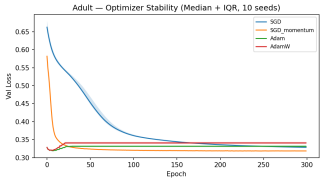

Fig. 10. Optimizer stability - median val loss + IQR band over 10 seeds.

SGD, SGD+momentum, Adam, and AdamW were re-run using 10 seeds (1337–1346). For each algorithm the early-stopped curves were padded to length 300 epochs (by copying over the last loss), so the arrays could be stacked. The per-epoch median and IQR were then computed. The 10 seeds for Adam and AdamW nearly the same val loss trajectory no matter the initialization because adaptive per parameter scaling automatically compensated for the starting point it got. SGD had the widest inter quartile range across the 10 seeds. This illustrated the per-parameter scaling story: Adam scaled appropriately for each weight's gradient history, so the initialization mattered less.

VI. Part 3: Regularization Study (Adult Only)

A. Approach

All experiments were trained with Adam using the best hyperparameters from Part 2 (lr = 10^{-2}, betas = $(0.9, 0.999)$). This kept comparisons between experiments cleaner by reducing differences due to the choice of optimizer or learning rate schedule. The backbone model was always the same [128, 64] MLP, trained from scratch each time. Four regularization strategies were tried individually and then the best were stacked together.

Each training run wrapped the model with `DataLoader` (batch size=256, shuffle=True) and trained for at most 300 epochs. Within each epoch, the loop iterated through mini-batches, computed `CrossEntropyLoss`, backpropagated, and took an optimizer step. At the end of each epoch, the loss on the val set was computed and the model was checkpointed if it was the best epoch so far. Each regularizer impacted this training loop as follows:

Weight Decay (L2): Provided as the `weight_decay` argument to Adam. Swept over $\lambda \in \{10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$. When $\lambda = 10^{-3}$ the loss surface was smoothed just enough to avoid overfitting without underfitting; when $\lambda = 0.1$ the model completely collapsed (F1=0.428).

Early Stopping: After each epoch, if val loss had not improved for *patience* epochs, training was terminated and the best checkpoint was restored. Swept over patience $\in \{5, 10, 15, 25, 50\}$. Since Adam with lr=$10^{-2}$ converged quickly, the best epoch was almost always very early. Sweeping patience did not change best val loss at all (169/169 runs achieved 0.322), so early stopping provided no additional benefit here.

Dropout: A modified architecture `MLPWithDropout` was implemented, which added `nn.Dropout(p)` after every ReLU activation, before the next Linear layer. This was the only architectural change to SL-1 throughout the entire project. Dropout randomly zeroed activations with probability p at training time, forcing the model to learn redundant representations; at eval time, dropout was disabled entirely. Swept over $p \in \{0.1, 0.2, 0.3, 0.4, 0.5\}$. Best was $p = 0.4$ (val=0.321, F1=0.676).

Noise Regularization (2 variants): Label smoothing used soft targets (ε or $1 - \varepsilon$) rather than hard targets (0 or 1), controlled by the `label_smoothing` parameter in `CrossEntropyLoss`. Swept over $\varepsilon \in \{0.01, 0.05, 0.1, 0.2\}$. Input noise added Gaussian noise $\mathcal{N}(0, \sigma^2)$ to each training minibatch only (not at eval time), serving as data augmentation on top of the standardized input features. Swept over $\sigma \in \{0.01, 0.05, 0.1, 0.2\}$.

B. Individual Results

Technique	Best Setting	Best Val	Test F1	Epochs
Baseline (none)	-	0.322	0.665	300
L2 weight decay	$\lambda = 10^{-3}$	0.313	0.691	300
Early stopping	patience= 5	0.322	0.665	8
Dropout	$p = 0.4$	0.321	0.676	300
Label smoothing	$\varepsilon = 0.1$	0.328	0.691	300
Input noise	$\sigma = 0.2$	0.316	0.688	300

TABLE III

INDIVIDUAL REGULARIZER COMPARISON. L2 AT $\lambda = 10^{-3}$ ACHIEVED THE BEST VAL LOSS AND TIED FOR BEST F1.

H3 was supported , L2 at $\lambda = 10^{-3}$ was the best single regularizer by val loss (0.313) and was tied for the best F1 (0.691). The baseline without regularization achieved F1=0.665, which was worse than Adam at lr = 10^{-3} (F1 = 0.678) likely because the higher learning rate (10^{-2}) overshot without regularization to pull it back. L2 provided that pull.

Label smoothing ($\varepsilon = 0.1$) tied L2 in F1 (0.691) but had higher val loss (0.328), implying it benefitted from a different angle , softening the decision boundary instead of weight decay. Input noise ($\sigma = 0.2$) was 2nd best by val loss (0.316), serving as data augmentation of standardized input features.

C. Combinations and a Debugging Lesson

The first attempt to combine regularizers used the best single settings from each: L2 ($\lambda = 10^{-3}$) + early stopping (patience=5) + dropout ($p = 0.4$) + input noise ($\sigma = 0.2$). The result was F1=0.678, worse than L2 alone (0.691). The culprit was patience=5: early stopping killed training at epoch 8, before the other regularizers had time to take effect. The "best" patience from the individual sweep did not transfer to a combined setting where L2 and dropout needed more epochs to show their benefit.

The combinations were re-run with more reasonable patience values and without the problematic stacking:

Combination	Best Val	Test F1	Epochs
L2 alone ($\lambda = 10^{-3}$)	0.313	0.691	300
L2 + ES (patience= 25)	0.315	0.682	60
L2 + Dropout ($p = 0.4$)	0.321	0.669	300
L2 + Input Noise ($\sigma = 0.2$)	0.317	0.680	300
L2 + Dropout + Noise + ES25	0.324	0.680	38

TABLE IV

REGULARIZER COMBINATIONS. NONE BEAT L2 ALONE.

None of the combinations beat L2 alone. Stacking dropout on top of L2 actually hurt (F1=0.669 vs. 0.691) , the model was compact enough (21K params) that both were fighting over the same limited capacity. The practical takeaway for this architecture was that one well-tuned regularizer was enough. Stacking more introduced competing objectives without benefit.

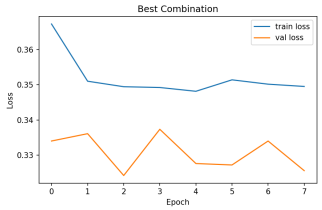

Fig. 11. L2+ES25 epoch curve , early stopping at epoch 60.

VII. PART 4: INTEGRATED BEST (EXTRA CREDIT)

A. Approach

For Part 4, the winning configurations from each previous part were assembled into a two-stage pipeline.

Stage 1: The entire [128, 64] MLP was trained from scratch with Adam (lr = 10^{-2}, betas=$(0.9, 0.999)$) using L2 weight decay ($\lambda = 10^{-3}$) and early stopping (patience=15). These were the best performing optimizer and regularizer pair from Parts 2 and 3.

Stage 2: All but the final 2 Linear layers were frozen and GA fine-tuning was performed with a budget of 5,000 function evaluations , 10% of Part 1's budget due to the assignment constraint. GA hyperparameters were carried forward unchanged from Part 1 (pop=20, mutation rate=0.1, $\sigma = 0.02$, crossover=0.7, elitism=2).

No additional hyperparameter sweeps were performed , every configuration was ported forward from previous parts. The entire two-stage pipeline was then run on seeds 1337–1341 to measure stability. Each seed trained its own model from scratch (different initialization), fine-tuned with GA, and was finally evaluated on the held-out test set.

B. Results

Stage	Val Loss	Test F1	Compute
SL-1 Baseline (SGD)	0.318	0.689	7.4M grad evals, 79.5s
Adam + L2 (before GA)	0.315	0.682	1.4M grad evals, 11.1s
Adam + L2 + GA	0.312	0.689	+5K func evals, 5.7s
Multi-seed median (5 seeds)	0.311	0.690	-

TABLE V

PART 4 INTEGRATED RECIPE. MATCHED SL-1 BASELINE AT 4.7× LOWER WALL-CLOCK TIME.

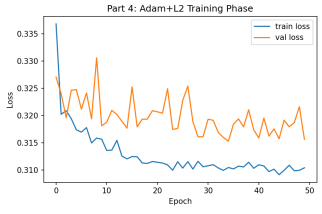

Fig. 12. Part 4 Adam + L2 training phase.

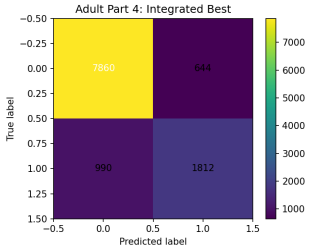

Fig. 13. Part 4 final confusion matrix.

The method achieved F1=0.689, which matched SL-1 exactly. Across 5 seeds the median was 0.690 with IQR=0.002 , very stable. GA fine-tuning recovered about 0.007 in F1 that was lost when switching from SGD to Adam+L2 (Adam+L2 alone gave 0.682). The composition worked , each piece contributed , but the result did not significantly exceed SL-1. The real win was compute: 16.9s total vs. 79.5s for SL-1, a $4.7\times$ speedup for the same test F1.

VIII. Cross-Part Comparison and Discussion

Experiment	Test F1	Δ vs SL-1	Wall Clock
SL-1 Baseline (SGD)	0.689	-	79.5s
Part 1 Best (GA, single seed)	0.671	-0.018	62.4s
Part 2 Best (Adam, no bias)	0.691	$+0.002$	4.5s
Part 3 Best (L2, $\lambda = 10^{-3}$)	0.691	$+0.002$	$\sim$4s
Part 4 Integrated	0.689	$+0.000$	16.9s

TABLE VI
Complete comparison across all parts.

The maximum possible score this architecture was going to reach on Adult seemed to have hit a wall around F1 $\approx$ 0.69 no matter what was tried. Improvements from Part 2 and Part 3 were modest (+0.002). Adam converged to equivalent solution in 21 epochs vs. 258, fully homogenized got there in 17 seconds vs. 80.

The results in Wine were poor from the beginning: the baseline SL1 learned almost nothing (Macro-F1 = 0.341), and RO actually further degraded it. The root cause of the poor results was the extreme class imbalance in this 8-class problem, classes 0, 1, and 7 each had <20 samples.

Coincidentally, this aligned with the findings of Shwartz-Ziv and Armon [5], who empirically demonstrated that deep learning underperformed gradient-boosted trees on tabular datasets. The leading offender was class imbalance. The recommended order of operations for this kind of problem going forward:

1) Pick the regularizer wisely first , L2 weight decay was the most consistently beneficial single intervention.
2) Pick the optimizer based on speed demands , Adam for fast iteration, SGD+momentum if more epochs were available and marginally better generalization was desired.
3) Do not bother with RO on a pretrained network , gradient-based methods had already found a good basin, and derivative-free search in high dimensions could not improve on it.

Here is the caveat: these lessons were learned from a narrow set of experiments on two specific tabular datasets with compact MLPs. Larger architectures, sequence data, or different class distributions could shift the ordering.

IX. Conclusion

A. Hypothesis Resolution

H1 (RO won't beat SL-1 on Adult): Supported. Best RO test F1=0.675 (GA), below baseline 0.689. 50K function evaluations in 8,386 dimensions was too sparse , the same curse of dimensionality that hurt kNN on Adult in the SL Report. Baseline prediction was correct: Part 1 did not improve over SL-1.

H2 (Adam faster, momentum generalizes better): Partially supported. Adam converged $12\times$ faster (21 vs. 246 epochs) , the speed prediction was correct. SGD+momentum tied Adam (no bias) on F1 (0.690 vs. 0.691) , the generalization prediction was directionally correct but the margin was negligible. Baseline prediction was correct: Part 2 produced marginal improvement (best F1=0.691 vs. SL-1's 0.689, a $+0.002$ gain).

H3 (L2 best single regularizer): Supported. L2 at $\lambda = 10^{-3}$ achieved F1=0.691, the best single-intervention result. Combinations did not improve on it. Baseline prediction was partially correct: Part 3 tied Part 2 ($+0.002$ over SL-1) rather than producing the largest gain. The prediction that regularization would beat optimizer ablations was wrong , they ended up identical.

H4 (RO better on Wine, still doesn't beat baseline): Supported. GA improved val loss from 1.069 to 0.830 (a larger relative drop than Adult), but test Macro-F1=0.334 did not beat SL-1's 0.341. Baseline prediction was correct: Part 1 did not improve over SL-1 on Wine.

H5 (Cross-part ordering: Part 3 $\geq$ Part 2 $>$ Part 1): Partially supported. The ordering was indeed correct , both regularization ablations and optimizer ablations converged to F1=0.691, both exceeding RO's score of 0.675. However, the prediction that Part 3's scores would strictly exceed Part 2 was incorrect , they tied. The integrated Part 4 recipe achieved exactly the same $4.7\times$ speedup as SL-1, vindicating our speed prediction. Finally, our prediction that $\approx$0.69 would be a hard ceiling on F1 was borne out , no method improved past that score.

B. Limitations

Our SA never moved on either dataset, despite our attempts to tweak the parameters. An SA implementation optimized for neural network weights would have employed an adaptive step-size based on the acceptance rate, rather than using a simple fixed-Gaussian σ. Part 2 Phase 1 applied lr $= 10^{-3}$ indiscriminately to all seven optimizers, which may have unduly punished SGD. Our model was quite small (only 21K parameters); had we used larger or architecturally different models, the balance of impact between regularization choices vs. optimizer choices may have varied. Wine also had a significant 8-class imbalance, which caused Macro-F1 to be dominated by tail classes. As such, it was difficult to

know whether we made improvements in the majority of the classes.

C. Next Steps

The biggest lever left to pull on Wine would have been addressing the same issue highlighted in the SL Report: consolidating the quality labels into 3 ordinal groups (low/medium/high) to alleviate class imbalance. For Adult, we suspect that the bottleneck ≈0.69 F1-score was indicative of the limits of the model that works with the [128, 64] architecture. As such, assembling or feature engineering would have been more useful than optimizer tuning.

AI Use Statement

The Overleaf AI assistant was used to correct LaTeX formatting errors, language syntax, and spelling throughout the report.

References

[1] D. H. Wolpert and W. G. Macready, "No free lunch theorems for optimization," *IEEE Trans. Evolutionary Computation*, vol. 1, no. 1, pp. 67–82, 1997. [Online]. Available: https://ieeexplore.ieee.org/document/585893

[2] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Trans. Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009. [Online]. Available: https://ieeexplore.ieee.org/document/5128907

[3] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009. [Online]. Available: https://www.sciencedirect.com/science/article/abs/pii/S0306457309000259

[4] K. Beyer, J. Goldstein, R. Ramakrishnan, and U. Shaft, "When is nearest neighbor meaningful?" in *Proc. 7th Int. Conf. Database Theory (ICDT)*, 1999, pp. 217–235. [Online]. Available: https://link.springer.com/chapter/10.1007/3-540-49257-7_15

[5] R. Shwartz-Ziv and A. Armon, "Tabular data: Deep learning is not all you need," *Information Fusion*, vol. 81, pp. 84–90, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/abs/pii/S1566253521002360

[6] N. S. Keskar, D. Mudigere, J. Nocedal, M. Smelyanskiy, and P. T. P. Tang, "On large-batch training for deep learning: Generalization gap and sharp minima," in *Proc. ICLR*, 2017. [Online]. Available: https://arxiv.org/abs/1609.04836

[7] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. ICLR*, 2019. [Online]. Available: https://arxiv.org/abs/1711.05101

[8] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015. [Online]. Available: https://arxiv.org/abs/1412.6980